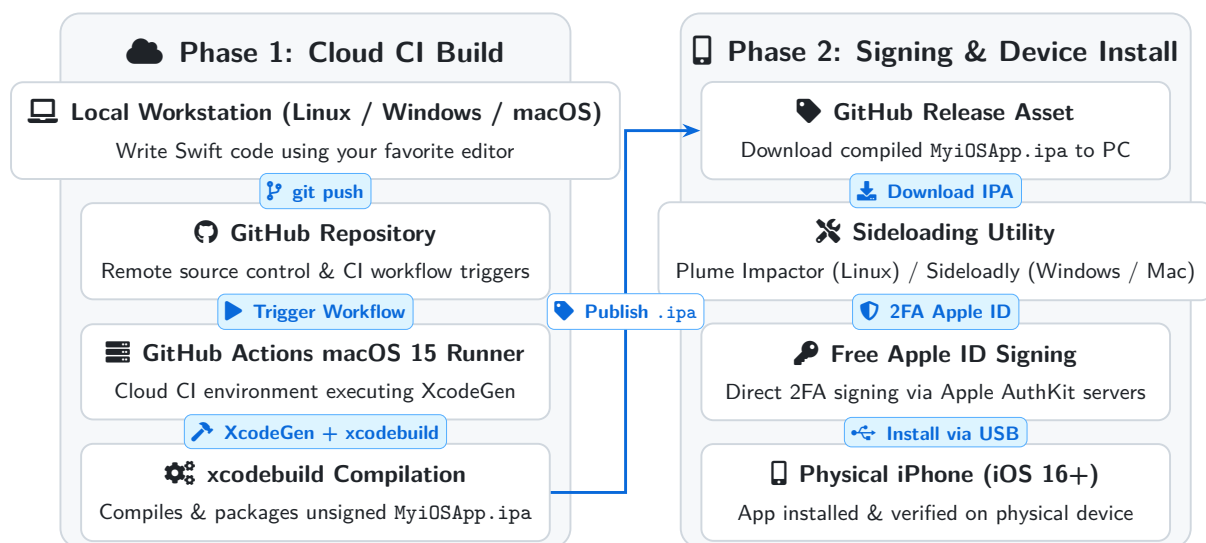


Cross-Platform Guide: Build iOS Apps with GitHub & Install Free on iPhone

A complete, beginner-friendly guide to building modern Swift iOS apps in the cloud using **GitHub Actions** and installing them on physical iPhones for **free** (no paid Apple Developer account or local Mac required).

Architecture Overview



Prerequisites

- An **iPhone** running iOS 16 or newer.
- A free **Apple ID** (your standard personal Apple account).
- A **GitHub** account.
- A computer running **Linux**, **Windows**, or **macOS**.
- A standard USB cable to connect your iPhone to your computer for the initial install.

Step 1: Clone Template, Initialize Git & Push

To create your own fresh iOS app using this template, clone this repository, re-initialize a clean git history, and push it to your GitHub account:

1. Clone & Re-initialize Git

```
# Clone this starter template
git clone https://github.com/Mehdirben/iosapp.git my-ios-app
cd my-ios-app

# Re-initialize a fresh git history for your project
rm -rf .git
git init
git add .
git commit -m "Initial commit: fresh iOS app template"
```

(Optional: Open *project.yml* to rename your app and bundle ID to whatever you like).

2. Push to Your GitHub

Actions Quota Considerations

Public repositories have **unlimited free GitHub Actions minutes**. Private repositories receive 2,000 free runner minutes/month. Each iOS build takes only ~1.5 minutes.

Choose whichever method you prefer:

Option A: One-Click GUI via VS Code (Easiest)

1. Open the project directory in VS Code (code `.`).
2. Switch to the **Source Control** tab in the sidebar (Ctrl+Shift+G or Cmd+Shift+G).
3. Click the “**Publish to GitHub**” button.
4. Select “**Publish to GitHub Public Repository**” or “**Publish to GitHub Private Repository**”.
5. VS Code will automatically create the remote repository on your GitHub account and push your code in a single click!

Option B: Via Terminal / CLI

1. Create a new repository on [GitHub](#) (Public or Private).
2. Link your repository and push:

```
git branch -M main
git remote add origin https://github.com/<your-username>/<your-repo-name>.git
git push -u origin main
```

Once pushed, GitHub Actions immediately begins compiling your app in the cloud!

Step 2: Download the Compiled IPA

1. On GitHub, navigate to the **Releases** section on the right sidebar of your repository.
2. Click on the latest release (e.g., **Build 1**).
3. Under **Assets**, click **MyiOSApp.ipa** to download it directly to your computer (no extraction required).

Step 3: Install the App on Your iPhone (Choose Your Platform)

Because the IPA is compiled in CI without Apple certificates, a sideloading utility signs it with your free personal Apple ID before installing it on your device.

🐧 Linux (Fedora, Ubuntu, Debian, Arch)

On Linux, use **Plume Impactor** (available directly on [Flathub](#)) — a modern, open-source GTK GUI tool that natively supports Apple's AuthKit 2-Factor Authentication (2FA) and free developer signing.

1. Install USB Device Communication Tools

- **Fedora / RHEL:**

```
sudo dnf install -y usbmuxd libimobiledevice libimobiledevice-utils
```

- **Ubuntu / Debian:**

```
sudo apt update && sudo apt install -y usbmuxd libimobiledevice6 libimobiledevice-utils
sudo systemctl start usbmuxd
```

- **Arch Linux:**

```
sudo pacman -S usbmuxd libimobiledevice
sudo systemctl start usbmuxd
```

2. Pair and Trust Your iPhone

1. Connect your iPhone via USB and unlock the screen.
2. Pair with Linux using the terminal:

```
idevicepair pair
```

3. A popup saying “**Trust This Computer?**” will appear on your iPhone screen. Tap **Trust** and enter your passcode.
4. Run `idevicepair pair` once more to confirm:

```
idevicepair pair
```

*(It will output: **SUCCESS: Paired with device <UDID>**).*

3. Install Impactor via Flathub (Recommended)

Install **Plume Impactor** directly from Flathub:

```
flatpak install -y flathub dev.khcrysalis.PlumeImpactor
```

```
# Grant permission to access local filesystem so you can drag-and-drop .ipa files easily:
flatpak override --user --filesystem=host dev.khcrysalis.PlumeImpactor
```

(Alternative for non-Flatpak systems: download and unpack the AppImage binary from [GitHub Releases](#)).

4. Install the App using Impactor

1. Ensure your iPhone is unlocked and connected via USB.
2. Launch **Plume Impactor** from your Applications menu (or run `flatpakrundev.khcrysalis.PlumeImpactor`).

Sequence Note

Always unlock and connect your iPhone **before** opening Impactor so it detects the device on launch.

3. Your connected iPhone will appear in the device selector.
4. Select or drag-and-drop your extracted `MyiOSApp.ipa`.
5. Sign in with your Apple ID and enter the 6-digit 2FA code sent to your device.
6. Click **Install**.

🖥️ Windows

On Windows, use [Sideloadly](#) — a beginner-friendly tool to sign and install apps on your iPhone.

1. Why iTunes & iCloud Are Required on Windows

Unlike macOS or Linux, Windows has no native drivers for iPhone USB communication, nor libraries to authenticate with Apple’s 2FA servers:

- **iTunes** provides Apple's official USB driver (`usbaapl64.sys`), allowing your PC to see and talk to the iPhone.
- **iCloud** provides Apple's authentication libraries (`ApplePushService.dll`), allowing Sideloadly to perform secure 2-Factor Authentication with Apple.

⚠️ Do Not Use the Microsoft Store Version

Microsoft Store apps run inside a locked, isolated sandbox. Sideloadly cannot access the drivers or files inside that sandbox. You **must** download and install the direct standalone installers from Apple using the links below:

1. **Download & Install iTunes (64-bit Windows):** [Direct Apple Download Link](#)
2. **Download & Install iCloud (Windows):** [Direct Apple Download Link](#)

(Restart your computer after installing if prompted).

2. Install Sideloadly

1. Download and install [Sideloadly \(64-bit Windows\)](#).
2. Connect your iPhone to your PC using your USB cable.
3. Unlock your iPhone screen. When prompted with “**Trust This Computer?**”, tap **Trust** and enter your passcode.

3. Sign & Install the App

1. Open **Sideloadly**. Your connected iPhone will automatically appear in the device dropdown at the top.
2. Drag and drop your extracted **MyiOSApp.ipa** into the large app icon box in Sideloadly.
3. Type your personal Apple ID email address into the **Apple ID** field.
4. Click **Start**.
5. When prompted, enter your Apple ID password and the 6-digit 2-Factor Authentication code that appears on your iPhone.
6. Sideloadly will sign the app with your Apple ID and install it directly to your iPhone home screen!

🍏 macOS

1. Download and install [Sideloadly](#).
2. Connect your iPhone to your Mac using a USB cable. Unlock the iPhone and tap **Trust** if prompted.
3. Open Sideloadly, drag and drop **MyiOSApp.ipa**, enter your Apple ID, and click **Start**.
4. Enter your Apple ID password and the 2FA verification code shown on your iPhone when prompted. Sideloadly will install the app directly via USB.

(Alternative for macOS: You can also use [iOS App Signer](#) + [Apple Configurator](#) / [Xcode](#)).

Step 4: First-Time iOS Device Settings

The first time you launch an app signed with a personal Apple ID, iOS blocks it until you authorize developer access. This is a one-time setup:

1. Trust Your Developer Certificate

1. On your iPhone, open **Settings** > **General** > **VPN & Device Management**.
2. Under **Developer App**, tap your Apple ID.
3. Tap **Trust** “[Your Apple ID]” and confirm.

2. Enable Developer Mode (iOS 16, 17, 18+)

1. Open **Settings** > **Privacy & Security**.
2. Scroll to the very bottom and tap **Developer Mode**.
3. Toggle it **ON** and tap **Restart**.
4. After your iPhone reboots, unlock the screen, tap **Turn On**, and enter your device passcode.

Your app is now ready to run!

Troubleshooting

Linux: iPhone not detected after restarting laptop or “Lockdown error -8”

If your iPhone is not detected after restarting your computer, check these items:

1. iPhone is Locked with Passcode (Before First Unlock)

After rebooting your computer or iPhone, iOS blocks all USB data transfer for security until you unlock the screen.

Rule

Unlock your iPhone screen with your passcode, then **unplug and reconnect the USB cable** once so iOS renegotiates the data connection.

2. Impactor Opened Before Phone Connected

Plume Impactor inspects USB devices once when it opens. If it was launched before you unlocked/plugged in your iPhone, it won't see it.

- **Fix:** Close Impactor and reopen it after connecting and unlocking your phone.

3. Root Cause: iPhone USB Hotspot Conflicts with usbmuxd (ipheth)

If Personal Hotspot is enabled (iOS may enable it automatically for trusted computers when cellular data is on), plugging in the iPhone brings up its USB-tethering interface. The Linux `ipheth` kernel driver claims it and NetworkManager activates it (an `enp...u5c4i2`-style USB-Ethernet interface appears) — and while that interface is coming up, it collides with `usbmuxd`'s device handshake: preflight fails with `lockdown error -8`, the phone starts disconnect/reconnect loops, and `usbmuxd` stops answering until restarted. This is why the phone is undetected at boot — or hours after boot whenever it is plugged in with hotspot on — yet works perfectly with hotspot off.

Confirm it is this case:

```
lsmod | grep ipheth           # tethering driver loaded
nmcli device | grep "c4i2"    # hotspot interface active
journalctl -u usbmuxd -b | grep "lockdown error -8" # failure signature
```

Fixes (pick one):

- **Simplest:** turn Personal Hotspot off (**Settings > Personal Hotspot**) before connecting the iPhone — `usbmuxd` then starts cleanly.
- **Permanent, if you never use USB tethering:** blacklist the `ipheth` driver so the tethering interface never activates. Wi-Fi hotspot is unaffected:

```
echo "blacklist ipheth" | sudo tee /etc/modprobe.d/blacklist-ipheth.conf
```

Delete that file to re-enable USB tethering later.

- **If you want both:** connect the iPhone with hotspot off (`usbmuxd` completes its handshake), then enable hotspot — both coexist fine afterwards.

4. usbmuxd Wedged Mid-Session (Manual Quick Fix)

If the phone is attached but invisible (typically after toggling the hotspot while plugged in, or reconnecting while locked), restart the daemon:

```
lsusb | grep -i apple # phone still physically connected?
idevice_id -l         # empty output = usbmuxd lost it
sudo systemctl restart usbmuxd
```

Test device communication anytime with:

```
idevice_id -l
```

(Your iPhone's 40-character UDID should print immediately).

5. Permanent Fix: Prevent usbmuxd Freezes & Automated Recovery

On Linux distributions (especially Fedora), two distinct issues can cause your iPhone to disappear from Impactor:

1. **USB-Tethering Conflict (ipheth):** When plugged in with cellular data/hotspot active, the Linux `ipheth` kernel driver clashes with `usbmuxd`, triggering an instant unbind event that freezes `usbmuxd` inside a `libusb` poll loop.
2. **Silent Daemon Hangs:** When `usbmuxd` deadlocks in memory, `systemd` still considers it active (`Active: running`), meaning standard process restart policies do not trigger.

Apply this complete 2-step permanent solution:

Step A: Blacklist ipheth (Prevents USB Collision)

```
echo "blacklist ipheth" | sudo tee /etc/modprobe.d/bl
sudo rmmod ipheth 2>/dev/null || true
```

Step B: Persistent Service & Automated Self-Healing Watchdog This configures `usbmuxd` to auto-start on boot and runs an automated timer that verifies socket responsiveness every 20 seconds, reviving the daemon if it ever freezes:

```
# 1. Ensure persistent service across boots
sudo mkdir -p /etc/systemd/system/usbmuxd.service.d
sudo tee /etc/systemd/system/usbmuxd.service.d/override.conf << EOF
[Unit]
Description=Socket daemon for the usbmux protocol used by Apple devices

[Service]
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
EOF

# 2. Install automated healthcheck watchdog script
sudo tee /usr/local/bin/usbmuxd-healthcheck.sh << EOF
#!/bin/bash
if ! timeout 2 /usr/bin/idevice_id -l > /dev/null 2>&1; then
    if lsusb | grep -qi "05ac:"; then
        systemctl restart usbmuxd
    fi
fi
EOF
sudo chmod +x /usr/local/bin/usbmuxd-healthcheck.sh

# 3. Create systemd timer for background watchdog
sudo tee /etc/systemd/system/usbmuxd-healthcheck.service << EOF
[Unit]
Description=Healthcheck and auto-recovery for usbmuxd
After=usbmuxd.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/usbmuxd-healthcheck.sh
EOF

sudo tee /etc/systemd/system/usbmuxd-healthcheck.timer << EOF
[Unit]
Description=Periodic healthcheck for usbmuxd (every 20s)

[Timer]
OnBootSec=30s
OnUnitActiveSec=20s

[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now usbmuxd
sudo systemctl enable --now usbmuxd-healthcheck.timer
```

iOS: “Untrusted Developer” Error when Opening App

Go to **Settings > General > VPN & Device Management**, select your Apple ID, and tap **Trust**.

iOS 16+: “Developer Mode Required”

Go to **Settings > Privacy & Security > Developer Mode**, toggle it **ON**, and restart your device. Upon reboot, confirm **Turn On**.

Apple Free Developer Rules & Limits

Rule	Limitation	Solution
Certificate Expiration	7 days	Re-sign/reinstall the app once a week using your sideloading tool from your computer.
Max Sideloaded Apps	3 active apps simultaneously	If you hit the limit, delete or deactivate an app in your sideloading tool.
App ID Registration	Up to 10 App IDs per 7 days	Keep the <code>bundleIdPrefix</code> consistent in <code>project.yml</code> .

Step 5: Developing & Adding Features

You do **not** need Xcode or a Mac to continue building your application:

- **Edit UI:** Modify `ContentView.swift` or create new SwiftUI views in `Sources/App/`.
- **Add Code Files:** Add any `.swift` files to `Sources/App/`. When GitHub Actions runs, **XcodeGen** automatically scans the folder and updates the project structure.
- **Deploy Changes:** Commit and push your changes:

```
git commit -am "Add new feature"
git push
```

GitHub Actions will compile the new `.ipa` automatically. Download and install the update!